Update Data with FastAPI

Now let's see how to update data in the database with a **FastAPI** *path operation*.

`HeroUpdate` Model

We want clients to be able to update the `name`, the `secret_name`, and the `age` of a hero.

But we don't want them to have to include all the data again just to **update a single field**.

So, we need to make all those fields **optional**.

And because the `HeroBase` has some of them *required* (without a default value), we will need to **create a new model**.

> **Tip**
>
> Here is one of those cases where it probably makes sense to use an **independent model** instead of trying to come up with a complex tree of models inheriting from each other.
>
> Because each field is **actually different** (we just set a default value of `None`, but that's already making it different), it makes sense to have them in their own model.

So, let's create this new `HeroUpdate` model:

Python 3.10+

```python
# Code above omitted 👆

class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
```

```
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

This is almost the same as `HeroBase`, but all the fields are optional, so we can't simply inherit from `HeroBase`.

Create the Update Path Operation

Now let's use this model in the *path operation* to update a hero.

We will use a `PATCH` HTTP operation. This is used to **partially update data**, which is what we are doing.

Python 3.10+

```
# Code above omitted 👆

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
```

```
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

We also read the `hero_id` from the *path parameter* and the request body, a `HeroUpdate`.

Read the Existing Hero

We take a `hero_id` with the **ID** of the hero **we want to update**.

So, we need to read the hero from the database, with the **same logic** we used to **read a single hero**, checking if it exists, possibly raising an error for the client if it doesn't exist, etc.

Python 3.10+

```
# Code above omitted 👆

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

👀 Full file preview

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
```

```
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

Get the New Data

The `HeroUpdate` model has all the fields with **default values**, because they all have defaults, they are all optional, which is what we want.

But that also means that if we just call `hero.model_dump()` we will get a dictionary that could potentially have several or all of those values with their defaults, for example:

```
{
    "name": None,
    "secret_name": None,
    "age": None,
}
```

And then, if we update the hero in the database with this data, we would be removing any existing values, and that's probably **not what the client intended**.

But fortunately Pydantic models (and so SQLModel models) have a parameter we can pass to the `.model_dump()` method for that: `exclude_unset=True`.

This tells Pydantic to **not include** the values that were **not sent** by the client. Saying it another way, it would **only** include the values that were **sent by the client**.

So, if the client sent a JSON with no values:

```
{}
```

Then the dictionary we would get in Python using `hero.model_dump(exclude_unset=True)` would be:

```
{}
```

But if the client sent a JSON with:

```
{
    "name": "Deadpuddle"
}
```

Then the dictionary we would get in Python using `hero.model_dump(exclude_unset=True)` would be:

```
{
    "name": "Deadpuddle"
}
```

Then we use that to get the data that was actually sent by the client:

Python 3.10+

```python
# Code above omitted 👆

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
```

```python
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

Tip

Before SQLModel 0.0.14, the method was called `hero.dict(exclude_unset=True)`, but it was renamed to `hero.model_dump(exclude_unset=True)` to be consistent with Pydantic v2.

Update the Hero in the Database

Now that we have a **dictionary with the data sent by the client**, we can use the method `db_hero.sqlmodel_update()` to update the object `db_hero`.

Python 3.10+

```python
# Code above omitted 👆

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
```

```
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

Tip

The method `db_hero.sqlmodel_update()` was added in SQLModel 0.0.16. 🤓

Before that, you would need to manually get the values and set them using `setattr()`.

The method `db_hero.sqlmodel_update()` takes an argument with another model object or a dictionary.

For each of the fields in the **original** model object (`db_hero` in this example), it checks if the field is available in the **argument** (`hero_data` in this example) and then updates it with the provided value.

Remove Fields

Here's a bonus. 🎁

When getting the dictionary of data sent by the client, we only include **what the client actually sent**.

This sounds simple, but it has some additional nuances that become **nice features**. ✨

We are **not simply omitting** the data that has the **default values**.

And we are **not simply omitting** anything that is `None`.

This means that if a model in the database **has a value different than the default**, the client could **reset it to the same value as the default**, or even `None`, and we would **still notice it** and **update it accordingly**. 🤯🚀

So, if the client wanted to intentionally remove the `age` of a hero, they could just send a JSON with:

```
{
    "age": null
}
```

And when getting the data with `hero.model_dump(exclude_unset=True)`, we would get:

```
{
    "age": None
}
```

So, we would use that value and update the `age` to `None` in the database, **just as the client intended**.

Notice that `age` here is `None`, and **we still detected it**.

Also, that `name` was not even sent, and we don't *accidentally* set it to `None` or something. We just didn't touch it because the client didn't send it, so we are **perfectly fine**, even in these corner cases. ✨

These are some of the advantages of Pydantic, that we can use with SQLModel. 🎉

Recap

Using `.model_dump(exclude_unset=True)` in SQLModel models (and Pydantic models) we can easily update data **correctly**, even in the **edge cases**. 😎